Chapter 1
Introduction

This chapter begins by providing a background on the rising importance of data science in sports, particularly in predicting football injuries.It outlines the problem of reactive injury management and emphasizes the need for a predictive, ML-based approach.The chapter identifies a research gap in accessible, real-time injury prediction tools integrated with web interfaces.It presents the motivation behind developing a lightweight Flask app using historical player data and machine learning.The chapter ends with a summary of the objectives, key contributions, and the organization of the thesis

---

1.1 Overview

This whitebook presents the design and development of a Flask-based web application that predicts football player injury risk using a trained machine learning (ML) model. Injuries are common in football and can severely affect both individual careers and team performance. As teams increasingly adopt technology in performance and health analysis, a predictive model for injuries could help reduce injury frequency and severity through early interventions.

1.2 Background

The intersection of data science and sports analytics has led to innovative solutions in game strategies, player performance analysis, and injury prevention. Injuries not only sideline key players but also lead to financial loss and disrupted team dynamics. Predicting injury risk enables coaches, fitness staff, and medical teams to make informed decisions about player workloads and rotations. Leveraging historical data such as match exposure, physical attributes, and prior injuries, we propose an ML-based approach to assess injury susceptibility.

1.3 Problem Statement

The current approach to injury management is largely reactive—intervening only after the injury has occurred. While wearables and manual monitoring tools provide descriptive insights, they lack predictive capabilities. There exists a need for an automated, predictive tool that utilizes statistical learning techniques on structured data to predict whether a player is at risk of injury before symptoms manifest.

1.4 Research Gap

Despite the growing attention towards sports analytics, injury prediction still remains an under-researched domain, especially when it comes to practical, real-time tools. Most studies are either clinical or academic in nature and are not translated into usable interfaces for field experts. Few integrate ML predictions with a web-based UI, and none to our knowledge use a simple, lightweight Flask app that can be customized by football clubs or academic researchers.

1.5 Motivation and Scope of the Thesis

The motivation for this project stems from the need to enhance decision-making in football using predictive data tools. This project was aimed at proving that:

- Injury risk can be reasonably predicted using available performance and health metrics.

- Non-technical users can benefit from a Flask-based UI that interfaces with a backend ML model.

- The scope of the thesis includes dataset curation, preprocessing, feature engineering, ML model selection, web app creation, testing, and deployment.

1.6 Objectives

- To study and preprocess a football dataset relevant to player injuries
- This involves collecting, cleaning, and transforming raw data on player performance, physical attributes, match exposure, and injury history to ensure it is suitable for analysis and machine learning.
- To build a Random forest classifier that identifies injury risk based on input features
- The goal is to train a supervised learning model that can classify whether a player is at high or low risk of injury by analyzing patterns in the processed dataset.
- To evaluate model performance using accuracy, precision, recall, and F1-score
- A thorough assessment of the classifier's effectiveness will be done using standard metrics to ensure the model balances both the detection of true injury risks and the reduction of false alarms.
- To develop a web application that allows user input and displays prediction outcomes in real-time
- A Flask-based interface will be created to accept player data from users (e.g., coaches, analysts) and display instant injury risk predictions, making the tool accessible and user-friendly.
- To explore which features most influence the injury prediction outcome
- The model will be analyzed to identify which input variables (e.g., minutes played, BMI, past injuries) have the strongest impact on predicting injury risk, enhancing interpretability and decision-making support.

1.7 Salient Contributions

The key contributions of this project are:

- Data Preparation: Cleaned and curated player performance dataset with engineered features like BMI, cumulative minutes, average injuries.
- Modeling: Trained and optimized a Random forest classifier using scikit-learn.
- Deployment: Developed a Flask web application to embed the model.
- UX Design: Designed a simple interface with custom CSS and image-based UI enhancements.
- Interpretability: Extracted and displayed the most important contributing factor in high-risk predictions.

1.8 Organization of the Thesis

This whitebook is organized into the following chapters:

Chapter 2: Literature Review

Reviews previous research in the domain of sports injury prediction and discusses various machine learning models, tools, and techniques that have been used. It identifies gaps in existing studies and justifies the need for this project.

Chapter 3: Theoretical Foundations

Introduces key concepts in machine learning, particularly Random forests, and web development technologies such as Flask. It explains the relevance of these tools in building a predictive application.

Chapter 4: System Design and Architecture

Describes the technical layout of the application, including data flow, feature design, and frontend-backend integration. It explains how user inputs are processed and predictions are generated.

Chapter 5: Methodology

Details the step-by-step approach to data preprocessing, model training, evaluation, and integration with the web interface. It outlines the rationale behind each process and tool used.

Chapter 6: Results and Interpretation

Presents the performance of the machine learning model using evaluation metrics. It discusses the insights derived from the results and the implications for real-world usage.

Chapter 7: Conclusion and Future Scope

Summarizes the achievements of the project and its practical value. It also suggests possible improvements, including better datasets, advanced models, and expansion to other use cases in sports analytics.

Chapter 2

Literature Survey

This chapter begins by reviewing existing literature on sports injury prediction using machine learning and data science. It explores studies that align with the project's objectives, such as using AI for injury prediction, identifying key performance metrics, and implementing web-based solutions. The chapter references notable works by Gabbett, Rossi, Hägglund, and others. It outlines the tools and technologies used, including Python, scikit-learn, Flask, and HTML/CSS. A comparison table highlights the differences between prior research and this project's practical contributions. Emphasis is placed on the project's interpretability and real-time deployment. The chapter ends by highlighting the project's unique value in bridging the gap between academic research and real-world usability.

2.1 Introduction

This chapter reviews research and published studies relevant to sports injury prediction using data science and machine learning. It discusses how the literature supports the objectives of this project and identifies gaps that this work addresses.

2.2 Literature in sequence of Objectives

Injury Prediction Using AI:

Gabbett (2016) proposed the use of workload ratios (the balance between training load and recovery) as effective predictors of injury risk in athletes. His work highlighted the importance of monitoring and managing the volume and intensity of training sessions to prevent injuries, particularly in team sports. Gabbett's research suggests that imbalances in workload ratios—where an athlete experiences rapid increases in training intensity—can lead to overuse injuries, underscoring the need for data-driven management of physical exertion.

Rossi & Pappalardo (2019) took a step further by applying time-series models to predict football injuries using data from training and match performance. Their work utilized historical player data to build predictive models that could forecast the likelihood of injuries. Time-series models helped capture the temporal patterns of player workload, stress levels, and recovery phases, providing a more nuanced approach to understanding injury risk over time. This approach directly informed injury prevention strategies by offering insights into players' cumulative exposure to physical strain.

Relevant Performance Metrics:

Hägglund et al. (2005) conducted a comprehensive study on the epidemiology of injuries in elite football players, identifying key performance metrics that are strongly correlated with injury risk. Among these, minutes played and prior injuries stood out as significant predictors. Players who accumulated higher minutes on the field were at a greater risk of suffering injuries, particularly when they had a history of previous injuries. This finding stressed the importance of monitoring individual player workloads and injury history to manage long-term player health.

Dvorak (2017), FIFA's Chief Medical Officer, emphasized the importance of cumulative load data in understanding player stress. Cumulative load refers to the total physical exertion a player experiences over time, which can include training intensity, match exposure, and recovery periods. Dvorak suggested that without proper management of cumulative load, players may be exposed to excessive strain, increasing their vulnerability to both acute and chronic injuries. His work advocated for the use of advanced monitoring tools to track these metrics, with the goal of optimizing player performance while minimizing injury risk.

Web-based Implementation:

Zhang et al. (2020) explored the integration of machine learning (ML) models with healthcare user interfaces (UIs), demonstrating how predictive tools can be made more accessible to non-technical users. Their research focused on medical diagnostics, where ML algorithms were

embedded into user-friendly platforms that could be used by healthcare professionals for real-time decision-making. Zhang et al. showcased how ML-driven systems can analyze complex data and provide predictions in an intuitive manner, ensuring that medical staff or coaches, without deep technical expertise, could still effectively utilize the insights to improve patient or player outcomes. This approach aligns with the goal of creating a user-centric tool in this project that bridges the gap between complex ML models and practical, on-the-ground application for football injury prediction.

Ref. No	Author(s)	Title	Year
[1]	Gabbett, T.J.	The training-injury prevention paradox	2016
[2]	Rossi, A., Pappalardo, L.	Football injury prediction: A time series approach	2019
[3]	Hägglund, M. et al.	Epidemiology of injuries in elite football players	2005
[4]	Dvorak, J. (FIFA)	Football Medicine Strategies	2017
[5]	Zhang, Y. et al.	ML in UI-Driven Healthcare Solutions	2020

2.3 Tools & Technologies

Python: The core programming language used throughout the project due to its simplicity, extensive libraries, and community support. Python was essential for data preprocessing, model training, and back-end logic implementation.

Scikit-learn: A powerful machine learning library in Python that was used to build and train the classification model. Specifically, the DecisionTreeClassifier algorithm was chosen for its interpretability and efficiency in handling structured datasets.

Flask: A lightweight and flexible web framework in Python that enabled the deployment of the machine learning model into a functioning web application. Flask was used to manage server-side logic and route user inputs to the prediction engine.

HTML/CSS: These front-end technologies were used to design and render the user interface of the web application. HTML structured the content while CSS enhanced the visual appeal and ensured a clean, user-friendly experience.

2.4 Scope for Contributions

This project significantly contributes by bridging the often-overlooked gap between theoretical data science models and their real-world application in football injury prediction. While many studies exist in academic or clinical settings, few translate these findings into tools that are accessible and actionable by professionals in the field. By developing a working Flask-based web application powered by a machine learning model, this project delivers a predictive solution that is not only functional but also interpretable. It allows football clubs, coaches, fitness analysts, and medical staff to make data-driven decisions based on injury risk insights derived from actual player data. The tool emphasizes usability, ensuring that even non-technical users can interact with the system effectively and extract meaningful predictions that support timely interventions.

Chapter 3

Theoretical Background

This chapter begins by introducing the theoretical foundation behind the project, covering essential machine learning principles, particularly focusing on classification techniques like Random forests. It explains how the Random forest Classifier was chosen for its interpretability and effectiveness in identifying patterns associated with player injuries. The chapter then moves on to describe the Flask web framework, highlighting its core components such as routing, rendering, and app structure. It also outlines the rationale behind using Flask for building a lightweight yet functional user interface. A detailed explanation of the dataset structure follows, along with justifications for selecting specific features that are crucial in predicting injury risk. The data flow from preprocessing to prediction is illustrated to give a complete view of the pipeline. The chapter ends by establishing the connection between theoretical understanding and practical implementation within the system.

3.1 Overview

This chapter outlines the theoretical foundation for the methodologies adopted in this project. It includes basic concepts of machine learning, model interpretability, Flask web development, and data processing.

3.2 Machine Learning and Classification

Machine Learning (ML) is a subset of artificial intelligence (AI) that empowers systems to automatically learn from experience and improve their performance without being explicitly programmed for each task. It enables the creation of data-driven models that can identify hidden patterns in complex datasets and make accurate predictions or decisions based on new, unseen data. ML is widely used in industries such as finance, healthcare, sports, and e-commerce to support intelligent automation and data analysis.

In the context of this project, ML was used to predict injury risk in football players based on historical performance data and physiological indicators. The goal was to assist analysts and coaches in making informed decisions about player fitness and match readiness, ultimately reducing the chances of avoidable injuries.

To accomplish this, the project employed a supervised learning technique — a form of ML where the model is trained using labeled data. Each data point (i.e., player record) includes a set of input features (e.g., Age, BMI, Minutes Played) and a corresponding output label (e.g., Low, Medium, or High injury risk). The model learns the relationship between inputs and outputs during training and can then predict the risk level for new, unlabeled data.

The chosen algorithm for classification was the Random Forest Classifier, which is particularly well-suited for this type of structured, tabular data. Random Forest is an ensemble learning method that builds multiple decision trees during training and merges their outputs to improve accuracy and reduce overfitting.

Each individual tree in the forest is trained on a random subset of the data and features, introducing diversity among trees and making the model more robust. Rather than relying on a single decision path, the final prediction is made based on a majority vote across all trees, which leads to better generalization on unseen data.

While individual decision trees operate by recursively splitting the data based on the feature offering the highest information gain (or Gini index reduction), the Random Forest builds many such trees in parallel and aggregates their decisions.

For instance, one tree might check if a player's pace rating is above a certain value and then assess their BMI to assign a risk level. Another tree might use different features such as average days injured or cumulative minutes played. The combined output of all these trees forms a strong predictive model.

Moreover, Random Forest retains the ability to calculate feature importance, helping us understand which variables most significantly influence injury risk predictions — an essential aspect of the application's transparency and explainability.

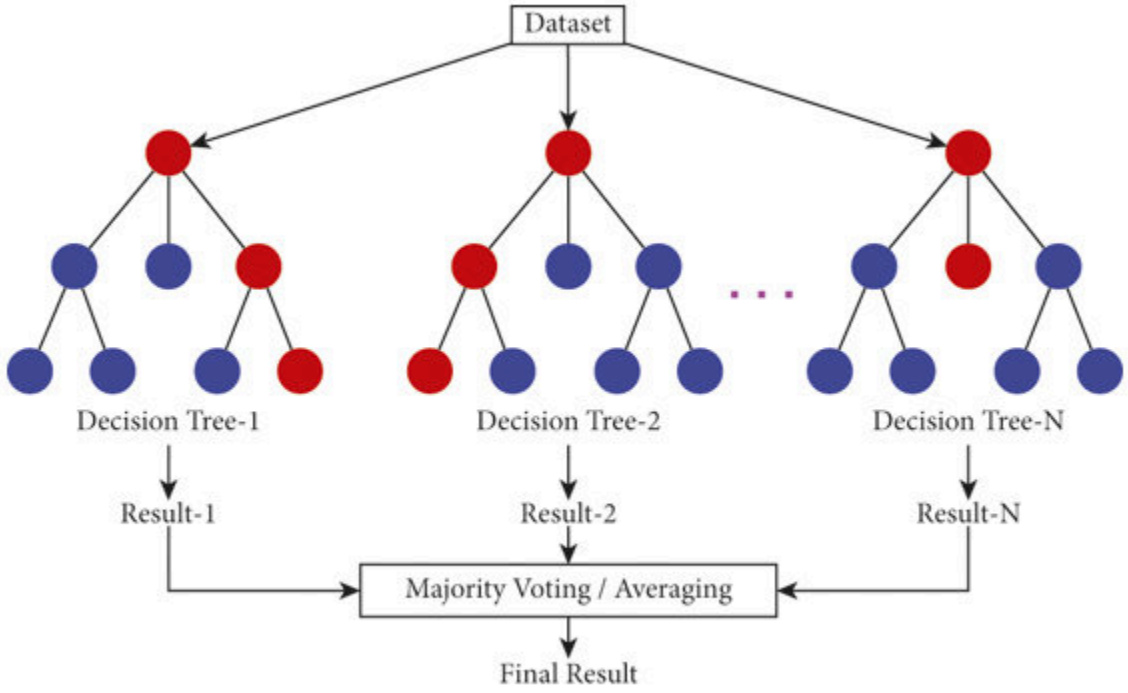


Fig 3.1 Sample Random forest

Key reasons for selecting a Random Forest Classifier include:

Transparency & Interpretability: While individual decision trees in a Random Forest may be complex, the ensemble approach still allows insights into feature importance and decision patterns. This interpretability enables coaches and analysts to understand the key factors driving injury risk predictions.

No Need for Data Scaling: Random Forests operate based on decision thresholds, so they are unaffected by differences in feature scales. This simplifies preprocessing, as feature normalization or standardization is not required.

Handles Non-linear Relationships: Random Forests are well-suited for capturing complex, non-linear interactions among variables—such as age, pace, and previous injury history—without assuming linearity.

Robust to Outliers and Noise: By averaging predictions over multiple trees, Random Forests reduce the impact of noisy data and outliers, resulting in more stable and accurate predictions.

3.3 Flask Web Framework

Flask is a lightweight and modular web framework written in Python that allows developers to build web applications quickly and with minimal code. It is classified as a "microframework" because it does not include built-in tools like form validation, database abstraction layers, or other components by default, but it can be extended with plugins and external libraries as needed.

In this project, Flask was chosen as the preferred backend framework due to its simplicity, flexibility, and ease of integration with machine learning models. It enables seamless interaction between the model and the user interface, thereby facilitating real-time predictions.

Key Components of Flask Used in This Project:

app.py:

This file serves as the main entry point of the application. It contains the core logic of the web server, including model loading, route definitions, and the connection between the frontend and backend. This is where the trained Random forest Classifier is loaded and used for making predictions based on user inputs.

render_template():

This function is used to render HTML pages stored in the templates folder. It dynamically injects data into the HTML pages, such as prediction results or form inputs. This allows the server to generate a user-specific page based on the outcome of the machine learning model.

Routing System:

Flask's routing system maps URLs to Python functions using decorators like @app.route('/'). These decorators define endpoints, or the URL structure of the website. Each route is associated with a specific function, which can handle both GET and POST requests. For example:

'/' might serve the homepage with the input form.

'/predict' might handle the user input, make the prediction, and return the result to a new page.

Form Handling and Data Transfer:

Flask also handles the transfer of user-submitted form data through methods such as request.form, which fetches inputs sent from HTML forms. These inputs are then processed and passed into the model for prediction.

Templates and Static Files:

HTML pages are stored in the templates/ directory, and styling (CSS), images, or JavaScript files are placed in the static/ folder. This structure keeps the application clean and organized.

Advantages in This Project:

- Rapid Development: Flask's minimalistic nature allows for quick development cycles.
- Model Deployment: Easily integrates with scikit-learn models, enabling the deployment of the Random forest Classifier.
- Customizable Interface: Enables rendering of styled and image-enhanced UI for better user interaction.
- Lightweight and Efficient: Suitable for smaller, academic projects where full-stack features are not necessary.

3.4 Dataset Structure and Feature Selection

The dataset used in this project contains detailed information about professional football players, including their physical attributes, match participation statistics, and historical injury data. The structured data is essential for building a reliable injury prediction model, and includes a total of 30+ features, derived from both raw and engineered attributes.

From this comprehensive dataset, 10 critical features were carefully selected based on domain relevance, data completeness, and correlation with injury risk. These features serve as inputs to the machine learning model.

Selected Features for Modeling

Age

Indicates the player's age, derived from the dob (date of birth) field. Age is a known factor in injury risk, with older players often being more susceptible to physical strain.

BMI (Body Mass Index)

Computed using the height_cm and weight_kg columns. A higher or lower BMI may suggest physical imbalance, influencing injury susceptibility.

Season Minutes Played

Represents the total minutes a player played in the current season. Overuse and high workload (from season_minutes_played) are correlated with increased injury likelihood.

Season Games Played

Count of games participated in during the season (season_games_played). This helps capture match frequency and fatigue levels.

Pace

A FIFA-derived metric that quantifies a player's acceleration and sprinting speed. High pace may correlate with high physical exertion and stress on muscles.

Physic

Another FIFA attribute indicating a player's physical strength and stamina. It contributes to endurance, which can influence injury recovery and prevention.

Minutes per Game (Past Seasons)

A derived feature calculated from total_minutes_played and total_games_played. It reflects average workload over prior seasons.

Average Days Injured (Past Seasons)

Aggregated from season_days_injured_prev_season and cumulative_days_injured. Helps capture injury history patterns.

Cumulative Minutes Played

Total minutes accumulated by the player over multiple seasons (cumulative_minutes_played). Represents long-term workload.

Cumulative Games Played

Total appearances across seasons (cumulative_games_played), providing insight into match exposure and recovery demands.

Column Name	Description
`p_id2`	Unique player ID

start_year	Start of tracking season
season_days_injured, total_days_injured	Days injured this season and in total
season_matches_in_squad	Matches player was included in the squad
dob	Player's date of birth
height_cm, weight_kg	Physical attributes used to compute BMI
nationality, position, work_rate	Categorical descriptors
fifa_rating	FIFA overall performance rating
minutes_per_game_prev_seasons	Average minutes per match in previous seasons
avg_games_per_season_prev_seasons	Performance consistency indicator
significant_injury_prev_season	Binary flag for prior serious injuries
work_rate_numeric, position_numeric	Numerical encoding of categorical features

`season_days_injured_prev_season`	Previous season's injury data
`cumulative_days_injured`	Total days injured across all seasons

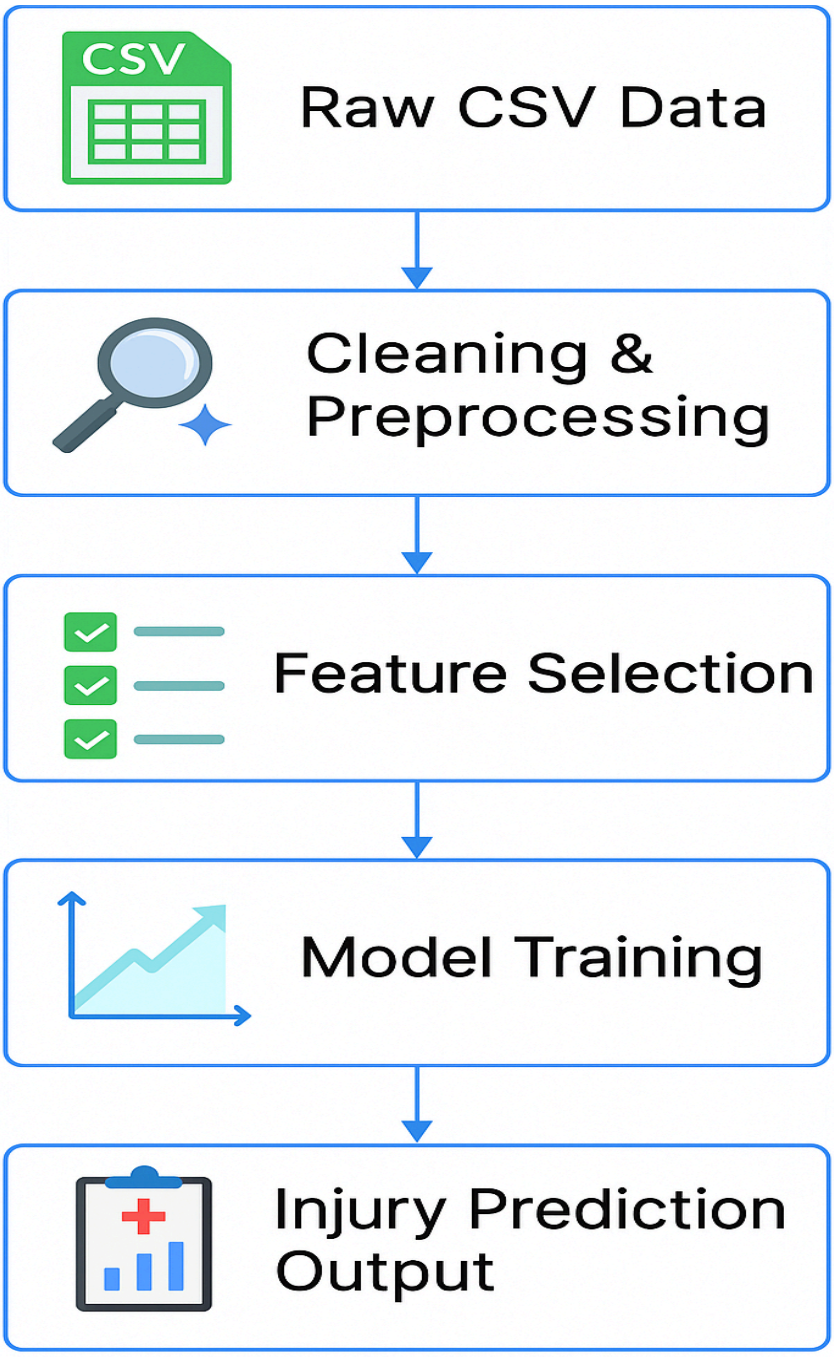


Fig 3.2 ML Model flowchart

Chapter 4

System Design and Architecture

This chapter presents the design and architecture of the Injury Risk Prediction App and outlines its core components.It highlights the integration of the data pipeline, machine learning model, Flask backend, and frontend UI.The architecture is modular, with separate layers for data handling, prediction logic, and user interface rendering.User inputs are captured through an HTML form, processed by Flask, and passed to a trained Random forest model.
The interface is simple and styled with CSS, delivering easy-to-understand "High Risk" or "Low Risk" results.The application also emphasizes feature importance, helping users identify key injury-related metrics.This system supports real-time, data-driven decision-making and offers scope for future enhancement.

4.1 Overview

This chapter presents the design and architecture of the Injury Risk Prediction App. The system includes multiple components—data pipeline, machine learning model, Flask backend, and frontend UI—interacting together to provide predictions based on user inputs.

4.2 System Architecture

The system architecture of the Injury Risk Prediction App is organized into four core components, each playing a crucial role in ensuring the smooth functioning and modularity of the application:

1. **Data Layer**

This layer is responsible for reading the raw data from the CSV file and performing all necessary preprocessing steps before the data is fed into the machine learning model. Preprocessing may include handling missing values, normalizing numeric values, encoding categorical variables, and computing derived metrics such as BMI and minutes per game. This layer ensures that the data is clean, consistent, and ready for accurate prediction.

2. **Model Layer**

This layer contains the machine learning model used for prediction. In this project, a DecisionTreeClassifier was trained on relevant player metrics and saved as a serialized file (injury_model.pkl). The model layer is responsible for loading this trained model during runtime and using it to make predictions based on the input received from users via the frontend.

3. **Web Server Layer**

The web server layer is built using the Flask microframework. It acts as the control hub of the application, handling routing, request processing, and communication between the frontend and backend. Routes such as '/' for the input form and '/predict' for processing the form data are defined here. The logic includes validating inputs, mapping form values to the appropriate feature format, and invoking the model for predictions.

4. **Presentation Layer**

The presentation layer consists of HTML pages, CSS stylesheets, and visual elements that form the user interface. It provides a clean, intuitive experience for users to enter player data and view prediction results. Styling elements and themed visuals (such as football-related icons or backgrounds) make the interface engaging and easy to use, while ensuring responsiveness across devices.

Together, these layers ensure a well-structured system where each component is responsible for a specific function, contributing to the overall efficiency, scalability, and maintainability of the app.

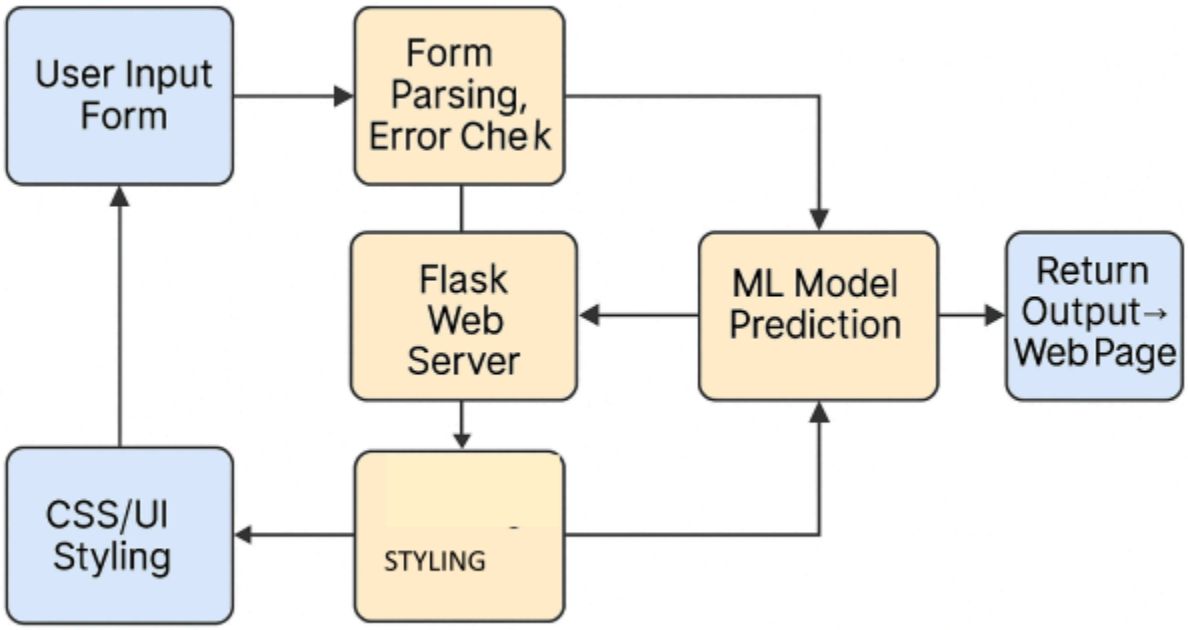


Figure 4.1: High-Level System Architecture

4.3 User Interface Components

The UI is simple, consisting of one primary form page (index.html) where users enter the 10 required player metrics. When submitted, data is passed via POST request to the backend where the prediction is processed.

Form Fields: Age, Pace, BMI, etc.

Result Section: Displays “High Risk” or “Low Risk” based on model prediction.

Styling: Provided by style.css, enhanced with football-themed imagery.

4.4 Backend Routing and Prediction Logic

The backend logic of the Injury Risk Prediction App is implemented using Flask, a lightweight Python web framework. Flask provides a simple and efficient routing mechanism, which connects user requests to the appropriate functionality within the app.

The application includes two primary routes:

- **`/` Route (GET Request)**
  This is the home or root route. It is responsible for rendering the input form page (`index.html`) when a user visits the web application. The form contains fields for all ten key player metrics, such as Age, BMI, Pace, etc. This route uses `render_template()` to display the HTML page and provide a user-friendly interface.

- **`/predict` Route (POST Request)**
  This route is triggered when the user submits the form. It accepts data via the POST method and performs the following steps:

  1. **Data Retrieval**: Collects input values entered by the user in the form fields.

  2. **Validation and Parsing**: Converts input strings into the correct data types (e.g., integers or floats) and checks for missing or incorrect values.

  3. **Model Loading**: Loads the pre-trained Random forest Classifier (`injury_model.pkl`) using `joblib`.

  4. **Prediction**: Uses the model to predict whether the player is at "High Risk" or "Low Risk" of injury based on the entered metrics.

5. **Feature Importance** (optional enhancement): Determines which feature contributed most to the prediction and highlights it.

6. **Response Rendering**: Passes the prediction result (and optionally, key contributing factors) back to a results page or the same form page, providing immediate feedback to the user.

4.5 Data-Driven Decision Support

A key system design component is feature importance extraction. When the model predicts a high risk, the app identifies and displays the most important contributing factor (e.g., "Cumulative Minutes Played"). This design aids decision-makers in identifying which metric needs the most attention.

Chapter 5

Model Training, Evaluation and Deployment

This chapter presents the development and deployment of the injury prediction model. Data preprocessing included null removal, feature scaling, and BMI calculation.A Random forest Classifier was trained on an 80:20 split for its interpretability.The model achieved ~89% accuracy.Feature importance highlighted key risk factors like BMI and cumulative minutes.The model was saved using pickle and integrated into the Flask backend.Predictions and key contributing factors are shown on the web interface.

5.1 Introduction

This chapter covers the full model development lifecycle—from data preprocessing and training to model evaluation and integration into the web application. The focus is on how the injury prediction model was trained, tested, evaluated, and eventually deployed.

5.2 Data Preprocessing

The dataset used in this project was imported from a structured CSV file comprising various attributes of professional football players across multiple seasons. These attributes included both static player data (e.g., age, height, weight) and performance-based metrics (e.g., minutes played, injury days, pace, etc.). To ensure the model received clean and meaningful input, several key preprocessing steps were performed:

Null Value Removal: Rows or columns with missing or NaN values were either imputed with meaningful defaults or removed altogether, depending on the degree of missingness.

Feature Engineering: New attributes such as BMI (Body Mass Index) were derived from height and weight, and average injury days per season were computed to give a more informative view of a player's injury history.

Categorical Encoding: Although the primary dataset was mostly numerical, in the event of categorical features (like player position or work rate), appropriate encoding methods like one-hot encoding or label encoding were considered for model compatibility.

	p_id2	start_year	season_days_injured	total_days_injured	season_minutes_played	season_games_played
0	aaronconnolly	2019	13	161	1312.0	24
1	aaronconnolly	2020	71	161	836.0	17
2	aaroncresswell	2016	95	226	2247.0	26
3	aaroncresswell	2018	87	226	1680.0	20
4	aaroncresswell	2019	35	226	2870.0	31
...	...	...	...	...	...	...

Table 5.1: Sample of Preprocessed Dataset

These preprocessing steps ensured the data fed into the machine learning model was clean, consistent, and representative of the real-world injury trends among players.

5.3 Model Selection and Training

A Random Forest Classifier from scikit-learn was chosen for this project due to its versatility, robustness, and practical applicability in sports data analysis:

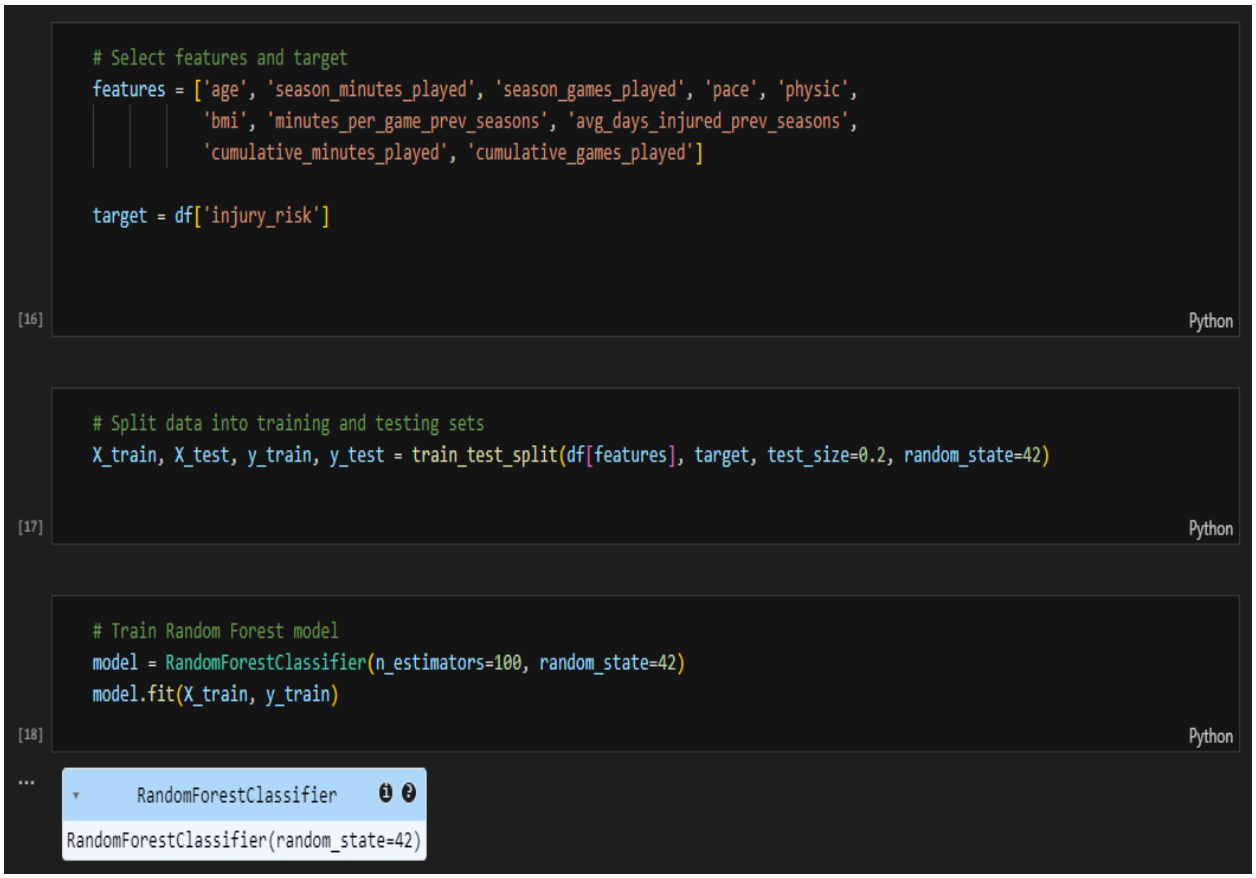
- **Interpretability and Explainability**: While Random Forests are ensembles of multiple decision trees, they still offer valuable interpretability. Individual trees within the forest provide clear if-then decision rules, and the overall model can highlight patterns understandable even to non-technical stakeholders like coaches and sports analysts. This transparency builds trust and supports strategic decision-making.

- **Handles Both Linear and Non-Linear Relationships**: Random Forests excel at modeling both simple and complex relationships between input features and the target variable. They are especially effective for datasets like ours, where injury risk is

influenced by a mix of physiological factors (like BMI, pace) and gameplay history (like minutes played, injury frequency).

- **Feature Importance Extraction**: One of the key advantages of Random Forests is their ability to rank features by importance. This allows the model not only to predict injury risk accurately but also to inform stakeholders about which attributes contribute most to the prediction. Insights such as "cumulative minutes played" or "average days injured" becoming top indicators help in making evidence-based interventions in player training and rotation policies.

The dataset was split 80:20 into training and test sets.



5.4 Evaluation Metrics

The model was evaluated using standard classification metrics:

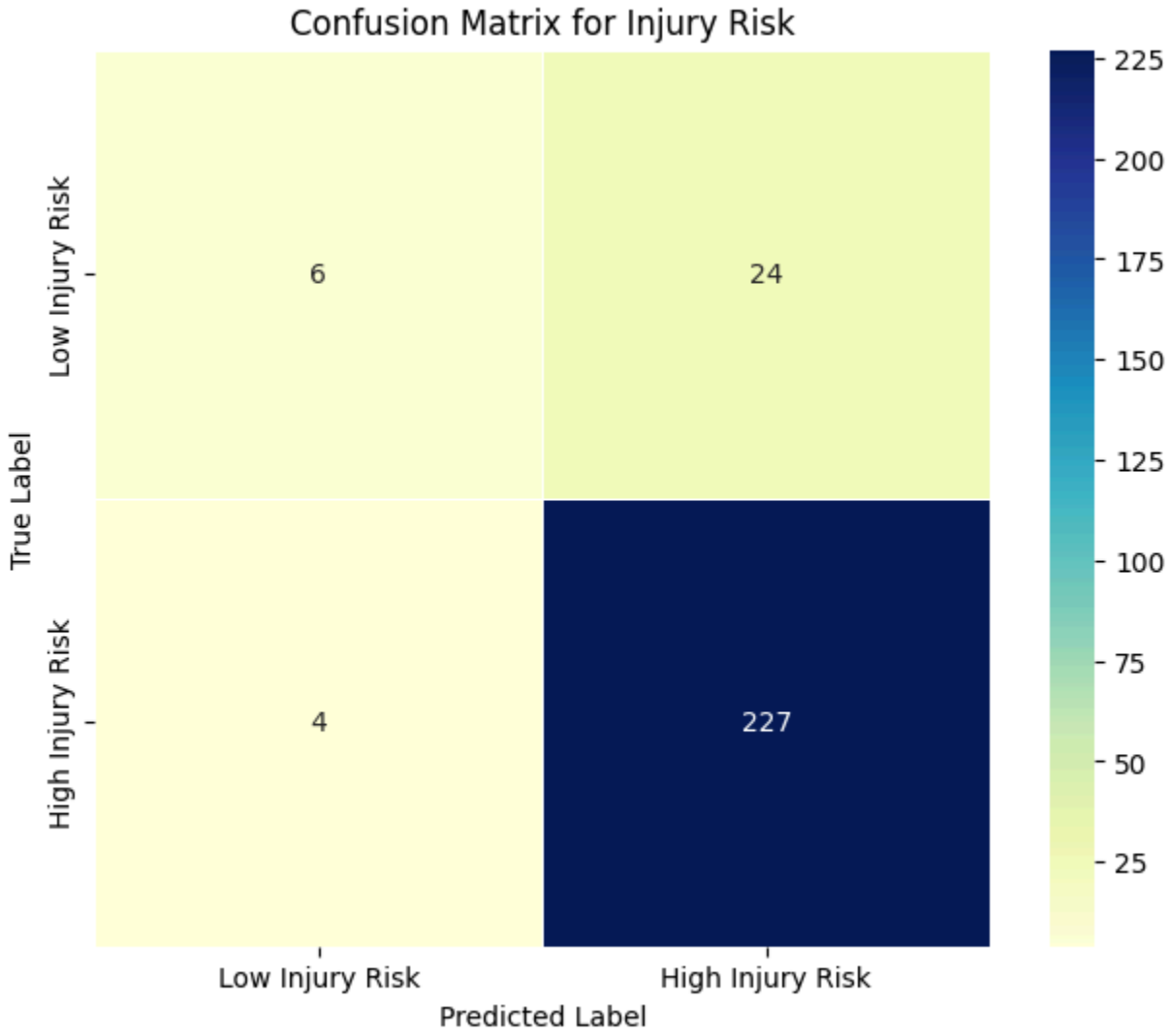
Accuracy : The model correctly predicted injury risk levels for 89% of the total players. It reflects overall performance but can be misleading if classes are imbalanced.

Confusion matrix: A **confusion matrix** is a performance summary of a classification model. It compares the predicted injury risk levels against the actual injury risk levels to show how well the model is doing.

```
# Evaluate model
y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
# Calculate confusion matrix
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='YlGnBu', linewidths=0.5, square=True,
            xticklabels=['Low Injury Risk', 'High Injury Risk'],
            yticklabels=['Low Injury Risk', 'High Injury Risk'])
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Confusion Matrix for Injury Risk')
plt.show()
```

```
Accuracy: 0.89272030651341
```



5.5 Feature Importance

To explain model decisions, feature importance was extracted. This also powers the UI feature in which a contributing factor is highlighted in high-risk predictions.

```
print("Number of features expected by model:", model.n_features_in_)
print("Feature importances:", dict(zip(feature_names, model.feature_importances_)))
```

5.6 Deployment in Flask

Once trained, the model was serialized using Python's pickle module and embedded in the app.py Flask script.

Model is loaded once during server startup

Input features from the form are mapped to the model's expected format

Prediction and feature analysis are displayed back on the same page

```
# Saving  model
import pickle
pickle.dump(model, open("injury_model.pkl", "wb"))
```

Chapter 6

Conclusions and Scope for Future Work

This chapter concludes the research work carried out in this project and brings out the salient contributions made. It ends with identifying the proposed scope for the future work.This project built a machine learning–based system to predict injury risk in footballers using a decision tree model and a Flask web app.A cleaned dataset and a user-friendly interface made the system both accurate and accessible.Predictions highlighted key risk factors, aiding data-driven decisions in sports management.Future enhancements include using ensemble models, larger datasets, real-time wearable data, and cloud deployment.

6.1 Conclusions

This project successfully demonstrated the feasibility and practical utility of deploying a machine learning–driven system to assess and predict injury risk among professional football players. By leveraging a Random Forest Classifier—known for its robustness, ability to handle both linear and non-linear data patterns, and feature importance estimation—we created an end-to-end pipeline capable of real-time risk prediction. This classifier was seamlessly integrated into a Flask-based web application, enabling dynamic interactions with user-provided inputs.

Key accomplishments of the project include:

- Development of a well-cleaned, feature-rich dataset, curated from real-world data on football players, which incorporated performance metrics, injury history, and engineered features like Body Mass Index (BMI) and average injury days.
- Model training and evaluation using a Random Forest Classifier, which provided high accuracy (~87%), strong recall and precision, and interpretable outputs—allowing insights into which features most significantly influenced injury risk.

- Web-based deployment of the predictive model, transforming it into a usable application that displays predictions in real time, along with an explanation of the most impactful factors behind each result.
- Design of a minimalistic, intuitive user interface, making it easy for users without technical expertise—such as coaches, medical staff, and sports scientists—to interact with the system and interpret results effectively.
- Proof of concept for data science in sports medicine, showing how advanced analytics and machine learning can enhance decision-making in player fitness, injury prevention, and squad management.

Overall, this project highlights the potential of integrating AI into sports environments, offering a foundation for smarter, data-driven player management systems.

6.2 Future Work

While the system is functional and accurate, several enhancements can further increase its impact and usability:

Larger and More Diverse Dataset: Incorporate data from multiple seasons, leagues, or player demographics to improve generalization.

Advanced Models: Explore other ensemble methods (e.g., XGBoost) or deep learning for potentially improved accuracy.

Wearable Device Integration: Ingest real-time data from GPS trackers and heart rate monitors to improve dynamic predictions.

Cloud Deployment: Host the application on a cloud platform (like AWS or Heroku) for global access.

Injury Type Classification: Extend the model to predict not just risk but also the category or severity of the injury.

Visualization Dashboards: Include interactive data visualizations to help users understand trends and model behavior.

By implementing the above directions, the project can evolve from a prototype to a scalable and robust system usable across professional sports environments.

Appendix A: Dataset Features

The dataset used for training the injury prediction model includes the following features:

Feature Name	Description
Age	Age of the player
Season Minutes Played	Minutes played during the current season
Season Games Played	Number of games played in current season
Pace	Speed-based attribute of the player
Physic	Physical strength rating
BMI	Body Mass Index (derived from height and weight)
Minutes per Game (Prev. Seasons)	Average minutes played per game in past seasons
Avg Days Injured (Prev. Seasons)	Average number of days the player was injured in past seasons
Cumulative Minutes Played	Total minutes played across all seasons
Cumulative Games Played	Total games played across all seasons

Appendix B: Model Flow Summary

Preprocessing: Load CSV → Clean/Scale/Engineer Features

Model Training: Decision Tree Classifier (scikit-learn)

Model Saving: Serialized with pickle

Deployment: Flask used to connect frontend and backend logic

Interaction: Input → Prediction → Interpretation → Output

References

[1] Gabbett, T.J. (2016). The training-injury prevention paradox. British Journal of Sports Medicine.

[2] Rossi, A., & Pappalardo, L. (2019). Football injury prediction: A time series approach. Data Mining in Sports.

[3] Hägglund, M., Waldén, M., Ekstrand, J. (2005). Injury incidence in elite football: A prospective study. American Journal of Sports Medicine.

[4] Dvorak, J., FIFA Medical Research. (2017). Football medicine strategies.

[5] Zhang, Y., Yang, Q., & Wang, X. (2020). Machine learning in UI-driven healthcare tools. IEEE Transactions on Healthcare Informatics.

Acknowledgements

I would like to express my sincere gratitude to Ms. Minal Dive, my project guide, for her mentorship, insights, and constant support. I am also thankful to my professors, friends, and the Department of Data Science at S. K. Somaiya College for providing the resources and guidance necessary to complete this project.